TEST AUTOMATION · AASTRIKA SPHERE

Sphere Web Automation

How we built it — achievements and the step-by-step work delivered across the account, course, competency, sign-up and course-page suites.

Prepared for: Aastrika / Sphere QA · **Platform:** Sunbird-iGOT · **Stack:** Playwright + TypeScript

Date: July 2026

Automated · CDP-authenticated · Bilingual (EN / HI)

Contents

1. What this project is
2. How it logs in (CDP)
3. Environments
4. The test suites & what we achieved
5. Reporting
6. How to run
7. Known limitation: yopmail
8. Achievements at a glance

01 · What this project is

An **automated testing suite** for the **Sphere** website — the Aastrika / Sunbird-iGOT nursing e-learning platform. Rather than a person clicking through the site by hand, these scripts drive a real browser: they log in, open and complete courses, fill forms, and verify every button and screen behaves as expected.

Tools: Playwright (browser automation) with TypeScript. **Coverage:** account/profile, course pages, course completion, competency assessments, sign-up, login, and the terms-and-conditions journey. **Output:** an Excel report and a Tarento-branded HTML report after every run.

02 · How it logs in — the CDP trick

Most pages require a logged-in session. Copying a session into the automation failed (the server rejects copied sessions), so we use **CDP (Chrome DevTools Protocol)**:

1. Open a Chrome window with remote debugging switched on.
2. Log in to Sphere once, by hand, in that window.
3. The tests **connect to that already-logged-in window** and drive it.

> **GOLDEN RULE**
> Always run from the `SphereWebAutomation` folder — never the parent `Automation` folder — or Playwright loads the wrong copy of itself and every test fails.

03 · Which site each test runs against

Test group	Runs against	Why
Account, Course, Competency, Course TOC	Production (logged-in CDP Chrome)	These pages render blank on the localhost mirror — they need the real site
Sign-up, Login, OTP, Positive Flow	Staging (`STAGING_URL`)	They create new accounts, which can't be done on localhost

Environments switch via two settings in `.env`: `BASE_URL` (logged-in pages) and `STAGING_URL` (sign-up flows).

04 · The test suites & what we achieved

Account page — 68 checks, all passing

Covers: user name & position, every left-nav link, all fields (name, phone, email, DOB, country, pincode), Organization & Academic sections, Language Preference, My Certificates, Logout, and rendering on mobile / tablet / desktop.

How we achieved it: fixed 21 broken checks to reach **68/68**; made every check work in **English and Hindi**; made the suite **open the Account page once and run all checks on that single session**; and added pop-up dismissal so one test can't block the next.

Sign-up (create-account)

Covers: pick language → fill form → verify by email OTP → confirm name → accept Terms.

How we achieved it: fixed a loading overlay that swallowed the OTP-submit click; **added a new journey** — create a user, click **Decline** on Terms (returns to the public page), **log in again**, then **Accept** the Terms; and throttled the email checks so the throwaway-email service doesn't bot-block us.

Course completion

Covers: plays every video to the end, answers quizzes/assessments (answer key + AI fallback), unlocks gated finals, waits for the Congratulations & Ratings pop-ups, and downloads the certificate.

How we achieved it: completed **"Care of Newborn & Newborn Resuscitation"** and **generated its certificate**; fixed the download by waiting for the certificate to finish drawing on screen before saving.

Competency — self-healing

Covers: the competency dashboard and completing assessment levels.

How we achieved it: if the dashboard shows "no result" (wrong role), the test auto-navigates to Account → Organization Details, sets **Background = Healthcare Worker** and **Designation = ANM/MPW**, saves, and reloads — then competencies appear. Broke it into **many per-step test cases**, uses the real **Job Description → role → Assess** path, navigates via the **Competency menu button** (a direct URL redirects home), and added a **Passbook** check.

Course TOC page — 16 skips turned into coverage

Covers: title, CNE hours, duration, rating, author, the Details / Certificate / Share icons, Resume/Start, the Overview / Modules / Discussion tabs, module list, Description, and License terms.

How we achieved it: the old tests skipped 16 of 22 by hunting for PDF/quiz viewers that live inside modules. We **rewrote them to check the real overview elements** — moving from 1

passing to **19 passing**. Found the real controls: the icons are `div.toc-icon-action` (click the box, not the label), the module list is `mat-list-item`, and the author is a coloured link.

Positive Flow — 26 clear steps

Covers: sign up → verify OTP → accept Terms → complete the profile (DOB, country/state/city, Healthcare Worker, designation, RN/RM number, organization type & name) → Done.

How we achieved it: turned one big test into **26 small test cases**, one per clicked element, so the report shows exactly how far the flow gets and where it stops.

05 · Reporting

Every run writes into three tidy folders and produces two report styles.

```
test-reports/
  ├─ excel-reports/     spreadsheet of every test (pass / fail / skip)
  ├─ course-reports/    course-completion reports
  └─ certificates/      downloaded certificates
brand-reports/          Tarento-styled HTML reports
```

Grouped runs merge into one report: `report:auth` (sign-up/login on staging) + `report:app` (account/course/competency on production) + `report:course` (completion) + `report:merge` (stitch together). A full combined run produced **149 passing tests**.

06 · How to run

From the `SphereWebAutomation` folder (open your logged-in Chrome on port 9222 first for the CDP suites):

```
npm run run:account          # Account page
npm run run:course-toc       # Course TOC page
npm run run:competency       # Competency
npm run run:login            # Login (staging)
npm run run:create-account   # Sign-up (staging)

# complete a course + get its certificate
$env:COURSE_NAME="Care of Newborn & Newborn Resuscitation"; npx playwright test tests/CourseCo

npm run report:sanity        # the full merged report
```

07 · Known limitation — yopmail

The sign-up tests read a one-time code (OTP) from a **throwaway email inbox (yopmail)**. When many sign-ups run quickly, yopmail flags us as a bot and shows a **CAPTCHA** that automation can't solve — so the OTP step fails and the sign-up flow stops there.

> NOT A CODE BUG
>
> The app *does* send the email (it returns "OTP sent"); we simply can't read it while yopmail is blocking us. We confirmed the OTP is **not** returned in the app's API response, so the email is the only source.

Options for later: (1) **wait it out** — the block clears in ~15–60 min; (2) **slower checks** — throttling is already added to stay under the bot limit; (3) **switch email provider** — a CAPTCHA-free service (e.g. 1secmail) as a last resort.

08 · Achievements at a glance

68/68	149	16→0	1
Account checks passing	Tests passing in the merged run	Course-TOC skips resolved	Certificate generated

Area	Delivered
Account	21 failures fixed → 68/68; bilingual EN/HI; single-session run
Sign-up	OTP-overlay fix; new Decline → re-login → Accept journey; email throttling
Course completion	Course completed end-to-end; certificate generated & download fixed
Competency	Self-healing role fix; per-step cases; Job-Description flow; Passbook check
Course TOC	Rewritten to real overview elements; 1 → 19 passing
Positive Flow	One test → 26 granular per-step cases
Reporting	Grouped + merged reports; correct environment routing
Diagnostics	yopmail CAPTCHA root-caused; throttling added

Everything is type-checked, and the Excel + branded reports generate automatically on each run. · Styled to the Tarento brand.